MODULE 3: DISTRIBUTED DATABASES

UNIT 4: DATABASE AND XML

3.4.1 Introduction

The use of eXtensible Markup Language (XML) is increasing in every field and it is imperative to have a secured place to store the XML documents. XML documents are stored in XML database. XML database is used to store huge amount of information in the XML format. The database can be queried using XQuery. XML is a markup language, which is mainly used to represent structured data. Structured data is the one which contains the data along with the tag / label to indicate what the data is. It is like a data with tag as a column name in RDBMS. Hence the same is used to document the data in DDB. One may think why we need to XML rather than simply documenting the data with simple tags as shown in the contact detail example. XML provides lots of features to handle the structured data within the document

3.4.2 Objectives

At the end of this unit, students should be able to:

4. Understand the various types of XML databases.
5. Understand and be able to state the differences between data-centric and document-centric XML.
6. To understand the principles of Web site management with XML

3.4.3 *What is XML?*

XML is an open and popular standard for marking up text in a way that is both machine and human readable. By “marking up text” we mean that the data in the text files is formatted to include meaningful symbols that communicate to a reader what that data is for. The syntax of XML is similar in style to HTML, the markup language of the World Wide Web (WWW).The data in an XML file can be organized into hierarchies so that the relationships between data elements are visually obvious.

1. XML is the markup language which serves the structured data over the internet, which can be viewed by the user easily as well as quickly.
2. It supports lots of different types of applications.
3. It is easy to write programs which process XMLs.
4. This XML does not have any optional feature so that its complexity can increase. Hence XML is a simple language which any user can use with minimal knowledge.
5. XML documents are created very quickly. It does not need any thorough analysis, design and development phases like in RDBMS. In addition, one should be able to create and view XML in notepad too.
6. All these features of XML make it unique and ideal to represent DDB. A typical XML document begins with <?xml..?>. This is the declaration of xml which is optional, but is important to indicate that it is a xml document. Usually at this beginning line version of the xml is indicated.

1. TYPES OF XML DATABASES

Native XML Database (NXD):

Native XML is a database that:

a. Defines a logical model for an XML document and stores and retrieves documents according to that model.

b. Has an XML document as its fundamental unit of (logical) storage, just as a relational database has a row in a table as its fundamental unit of (logical) storage.

c. The storage model itself is not constrained. For example, it can be built on a relational, hierarchical, or object-oriented database, or use a proprietary storage format such as indexed, compressed files.

2. XML Enabled Database (XEDB):

A database that has an added XML mapping layer provided either by the database vendor or a third party. This mapping layer manages the storage and

retrieval of XML data. Data that is mapped into the database is mapped into application specific formats and the original XML meta-data and structure may be lost. Data retrieved as XML is NOT guaranteed to have originated in XML form. Data manipulation may occur via either XML specific technologies (e.g. XPath, XSLT and DOM) or other database technologies (e.g. SQL). The fundamental unit of storage in an XML Enabled Database is implementation dependent.

7. Hybrid XML Databases (HXD):

A database is that can be treated as either a Native XML Database or as an XML Enabled Database depending on the requirements of the application.

Data-Centric Vs. Document-Centric Xml

Deciding whether to use an XML-enabled, native or hybrid XML database is in many cases difficult, and usually depends both on the specific application and the format of the XML documents.

Data-Centric Documents: Data-centric are documents produced as an import or export format, that is, data-centric XML documents are used for machine consumption. These documents are used for communicating data between companies or applications and the fact that XML is used as a common format is simply a matter of convenience, for reasons of interoperability Examples of data-centric documents are sales orders, scientific data, and stock quotes. Since data-centric documents are primarily processed by machines, they have fairly regular structure, fine-grained data and no mixed content.

Document-Centric Documents:

Document-centric are documents usually designed for human consumption, with examples ranging from books to hand-written XHTML documents. They are usually composed directly in XML, or some other format and then converted to XML. Document-centric documents do not need to have regular structure, have coarse-grained data (that is the smallest independent data unit may as well be a document itself) and have mixed content. The examples given above make it clear that the ordering of elements in such documents is always significant.

Storing XML in an XED

XML-enabled databases are primarily used in settings where XML is the format for exchanging data between the underlying database and an application or another database, for example when a Web service is providing data stored in a relational database as an XML document. The main characteristic of the XML-enabled database storage methodology is that it uses a data model other than XML, most commonly the relational data model. Individual instances of this data model are mapped to one or more instances of the XML data model, for example a relational schema can be mapped to different XML schemas depending on the structure of the XML document required by the application that will consume it. As shown in Fig.3.4.1, the XML documents are used as a data exchange format without the XML document to have any particular identity within the database. For example, suppose XML is used to transfer temperature data from a weather station to a database. After the data from a particular document is stored in the database, the document is discarded. To the opposite direction, when an XML document is requested as a result of a query, it is constructed from the results that are retrieved after querying the underlying database, and once it is consumed by the client that has requested it, it is again discarded. There is no way to ask explicitly for a document by its name, nor does any guarantee exist that the original document that was stored in the database can be reconstructed. Because of this, it is not a good idea to shred a document into an XML-enabled database as a way of storing it. The basic use

of XML-enabled systems is for publishing existing relational data (regardless of its source) as XML.

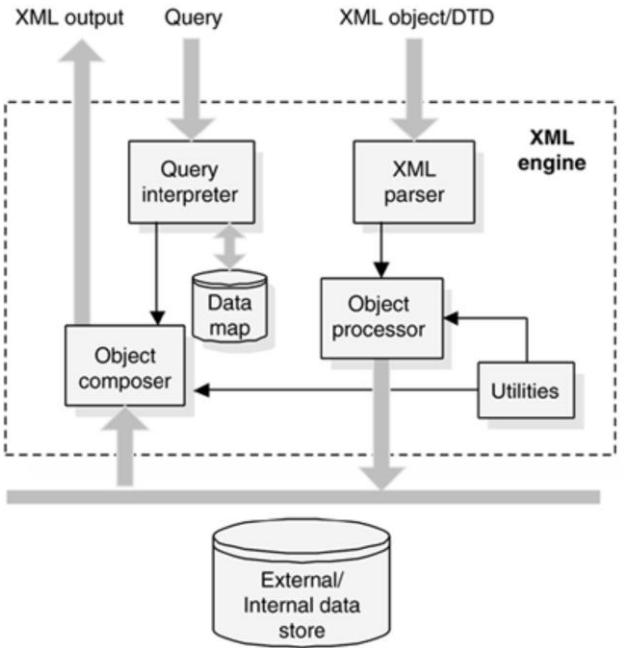

Figure 3.4.1: structure of the XML document

Regardless of whether we are shredding or publishing XML documents, there are two important things to note here. First, an XML-enabled database does not contain XML, i.e. the XML document is completely external to the database. Any XML document/fragment is constructed from data already in the database or is used as a source of new data to store in the database. Second, the database schema matches the XML schema, that is, a different XML schema is needed for each database schema. XML-enabled databases generally include software for performing both publishing and shredding between relational data and XML documents. This extra piece of software can either be integrated into the database or be provided by a third-party vendor outside the database. This software, used by XML-enabled databases, cannot, generally, handle all possible XML documents. Instead, it can handle the subclass of documents that are needed to model the data found in the database.

XML Data Models:

A data model is an abstraction which incorporates only those properties thought to be relevant to the application at hand. As there are different ways in which one can use XML data, there are more than one XML data models. As shown Fig 3.4.2, all XML data models include elements, attributes, text, and document order, but some also include other node types, such as entity references and CDATA sections, while others do not.

Therefore, DTD defines its own XML data model, while XML Schema uses the XML InfoSet data model; XPath and XQuery define their own common data model for querying XML data. This excess of data models makes it difficult for applications to combine different features, such as schema validation together with querying. The W3C is therefore in the process of merging these data models under the XML InfoSet, which is to be replaced by the Post-Schema-Validation Infoset (PSVI).

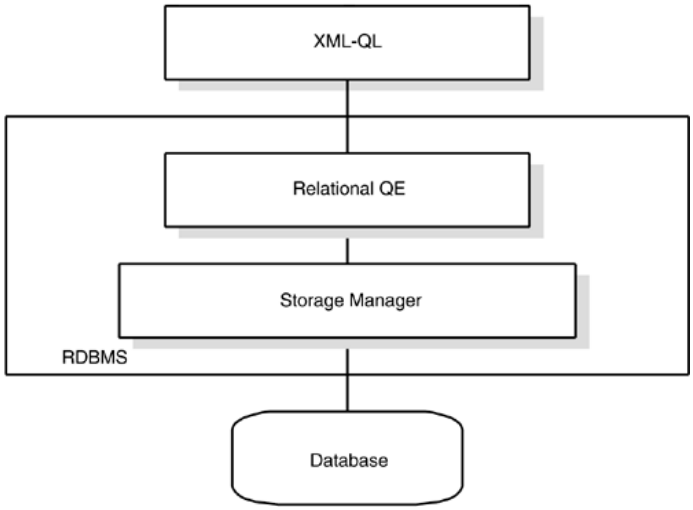

Fig. 3.4.2: Architecture of the XML-Enabled Database

Since many NXDs were created prior to the XML InfoSet and the XPath 2.0 and XQuery data model, they were free to define their own data model. Since the data model of an XML query language defines the minimum amount of information that a native XML database must store, these NXDs need to be upgraded to support XQuery.

Fortunately, the vast majority of NXDs currently supports at least XPath 1.0, and it is envisaged that all of them will support XQuery in the near future.

Relational-to-XML Schema Mapping

When using an XML-enabled database, it is necessary to map the database schema to the XML schema (or vice versa). Such mappings are many-to-many. For example, a database schema for sales order information can be mapped to an XML schema for sales order documents, or it can be mapped to an XML schema for reports showing the total sales by region. There are two important kinds of mappings:

a) Table-based mapping

b) Object-relational mapping

Both table-based mapping and object-relational mappings define bi-directional mappings, that is, the same mapping can be used to transfer data both to and from the database.

Table-Based Mapping:

When using a table-based mapping, the XML document must have the same structure as a relational database. That is, the data is grouped into rows and rows are grouped into \tables". In the following example, the SalesOrders and the Items elements represent the corresponding relational tables containing a list of SalesOrder and Item elements, respectively that in turn represent the rows of each table.

```
<Database>
<SalesOrders>
<SalesOrder>
<Number>123</Number>
<OrderDate>2003-07-28</OrderDate>
<CustomerNumber>456</CustomerNumber>
</SalesOrder>
</SalesOrders>
<Items>
<Item>
<Number>1</Number>
<PartNumber>XY-47</PartNumber>
<Quantity>14</Quantity>
<Price>16.80</Price>
```

```
<OrderNo>123</OrderNo>
</Item> <Item>
<Number>2</Number>
<PartNumber>B-987</PartNumber>
<Quantity>6</Quantity>
 <Price>2.34</Price>
<OrderNo>123</OrderNo>
</Item>
</Items>
 </Database>
```

Object-Relational Mapping:

When using an object-relational mapping, an XML document is viewed as a set of serialized objects and is mapped to the database with an object relational mapping. That is, objects are mapped to tables, properties are mapped to columns and inter-object relationships are mapped to primary key / foreign key relationships. Below is the same document containing sales orders, as above, but encoded using the object-relational mapping. <Database>

```
<SalesOrder>
<Number>123</Number>
<OrderDate>2003-07-28</OrderDate>
<CustomerNumber>456</CustomerNumber>
<Item>
<Number>1</Number>
<PartNumber>XY-47</PartNumber>
<Quantity>14</Quantity>
<Price>16.80</Price>
</Item>
<Item>
<Number>2</Number>
<PartNumber>B-987</PartNumber>
<Quantity>6</Quantity>
<Price>2.34</Price>
</Item>
</SalesOrder>
 </Database>
```

Retrieving and Modifying Query Languages

Query languages in XML-enabled databases are used to extract data from the underlying database and transform it. The most widely used query languages for this purpose, *SQL/XML* and *XQuery*. For XML-enabled relational databases, the most widely used query language is SQL/XML, which provides a set of extensions to SQL for creating XML documents and fragments from relational data and is part of the ISO SQL specification of 2003. The main features of SQL/XML are the provision of an XML data type, a set of scalar functions, XMLELEMENT, XMLATTRIBUTES, XMLFOREST, and XMLCONCAT, and an aggregate function, XMLAGG. For example, the following call to the XMLELEMENT function:

XMLELEMENT (NAME Customer,
XMLELEMENT (NAME Name, customers.name),
XMLELEMENT (NAME ID, customers.id))
constructs the following Customer element for each row in the customers table:
<Customer>
<Name>customer name
</Name>
<ID>customer id</ID>
</Customer>

Updating Xml-Enabled Databases

The solutions related to updating XML data in XML-enabled databases vary from implementation to implementation, due to the idiosyncrasies of the storage model. The most common practice is either to use a custom extension of the SQL/XML prototype to support updates or use a, custom again, product specific API to perform the modification operations over the stored XML data. In either case, though, the update action does not happen in-place, i.e. directly on the tables where the XML data is stored, but rather the document to be updated is extracted from the database, loaded in memory, updated, then returned to the XML-enabling software layer where it is shredded again and stored back to the database.

Fundamental Unit of Storage

A fundamental unit of storage is the smallest grouping of data that logically file together in storage. From a relational database perspective, the fundamental unit of storage is a tuple. From a native XML database perspective, the fundamental unit of storage is a document. Since any document fragment headed by a single element is potentially an XML document, the choice of what constitutes an XML document is purely a design decision.

Text-Based Native Xml Databases

Text-based native XML databases store XML as text. This may be a file within the database itself, a file in the file system outside the database, or a proprietary text format. Note here that the first case implies that relational databases storing XML documents within CLOB (Character Large Object) fields are considered native. All text-based NXDs make use of indices, giving them a big performance advantage when retrieving entire documents or document fragments, as all it takes is a single index-lookup and a single read operation to retrieve the document. This is contrary to XML-enabled databases and some model-based NXDs which require a large number of joins in order to recreate an XML document that has been shredded and inserted in the database. This makes the comparison between text-based NXDs and relational databases analogous to the comparison between hierarchical and relational databases, in that text-based NXDs will outperform relational databases when returning the document or document fragments in the form in which the document is stored; returning data in any other form, such as inverting the document hierarchy, will lead to performance problems.

Model-Based Native Xml Databases

Model-based NXDs have an internal object model and use this to store their XML documents. The way this model is stored varies: one way is to use a relational or object-oriented database; other databases use proprietary storage formats, optimized for their chosen data model. Model-based NXDs built on other databases will have performance similar to those databases, since they rely on these systems to retrieve data; the data

model used, however, can make a notable difference in performance. Model-based NXDs using a proprietary storage format usually have physical pointers between nodes and therefore are expected to have similar performance to text-based NXDs. Clearly, the output format is significant here, as text-based NXDs will probably be faster in outputting in text format, whereas a model-based NXD using the DOM model will be faster in returning a document in DOM format.

Attributes are used to give more meaning to the data represented within the elements. Here attribute indicates what type of contact details are listed like address, phone, email etc.

```
<Contact  category=”ADDRESS”>
<Name> Rose Mathew </Name>
<ApartmentNum>APT 201 </ ApartmentNum>
<AppName> Lakeside terrace 1232 </AppName>
<Street>Lakeside Village Drive </Street>
<Town> Clinton Township </Town>
<State> MI </State>
<Country> US </Country>
</Contact>
```

These elements, attributes are all known as nodes in the document. In short, nodes are the tags / labels in the document. There are seven types of nodes in the XML documents.

1. **Root** : This is the beginning of all the nodes in the document. In our example above contact is the root node.

```
<Contact >
```

2. **Element :** This is the any node in the document that begins with <name> and ends with </name>.

```
<ApartmentNum>APT 201 </ ApartmentNum>
<AppName> Lakeside terrace 1232 </AppName>
```

3. **Text :** This is the value of the element node. In below example, 'Rose Mathew' is a text node.

```
<Name> Rose Mathew </Name>
```

4. **Attribute :** This is the node within the beginning element of the document which specifies more details about the element. It contains name and its value pair always.

```
<Contact  category="ADDRESS">
```

5. **Comment :** This node contains the comment or the description about the data, element or attribute or anything. But it has nothing to do with the actual data. Its only for understanding the document. It is starts with <!-- and ends with --->.

```
<!-- This is the comment node -->
```

6. **Processing Instruction :** This is the node which gives the instruction to the document like sort, display, or anything to do with document. It is always a child node beginning with <? and ending with ?>.

```
<?sort alpha-ascending?>
  <?StudentNames <Fred>, <Bert>, <Harry> ?>
```

7. **Namespace :** Namespace indicates to which bucket the elements belong to. For example, there would same element names used in the document which will have different meaning in their contest – state in address and state for STD code. In order to differentiate this we use **namespace.**

```
<Address: State>
<Phone: State>
```

8. Now it is clear what each state is for. This is similar to appending table name or its alias name before the column names in SQL.

9. These are the very basic things that we need to know while creating an xml document. Let us see how it can be applied in DB.

1. XML elements

10. XML elements can be defined as building blocks of an XML. Elements can behave as containers to hold text, elements, attributes, media objects or all of these. Each XML document contains one or more elements, the scope of which are either delimited by start and end tags, or for empty elements, by an empty-element tag.

Syntax

The following is the syntax to write an XML element –

```
<element-name attribute1 attribute2>
....content
</element-name>
```

where,

1. **element-name** is the name of the element. The *name* its case in the start and end tags must match.
2. **attribute1, attribute2** are attributes of the element separated by white spaces. An attribute defines a property of the element. It associates a name with a value, which is a string of characters. An attribute is written as −

   name = "value"

name is followed by an = sign and a string *value* inside double(" ") or single(' ') quotes.

Empty Element

An empty element (element with no content) has following syntax −

```
<name attribute1 attribute2.../>
```

Following is an example of an XML document using various XML element −

```
<?xml version = "1.0"?>
<contact-info>
   <address category = "residence">
```

```
      <name>Tanmay Patil</name>
      <company>TutorialsPoint</company>
      <phone>(011) 123-4567</phone>
   </address>
</contact-info>
```

XML Elements Rules

Following rules are required to be followed for XML elements −

1. An element *name* can contain any alphanumeric characters. The only punctuation mark allowed in names are the hyphen (-), under-score (_) and period (.).
2. Names are case sensitive. For example, Address, address, and ADDRESS are different names.
3. Start and end tags of an element must be identical.
4. An element, which is a container, can contain text or elements as seen in the above example.

3.4.6 Self-Assessment Question

Is XML a programming language? State reasons to justify your answer.

1. Summary

XML is actually a language used to create other markup languages to describe data in a structured manner. It is a widely supported open technology, (that is a non-proprietary technology) for data exchange and storage.

3.4.7. Conclusion

Processing an XML document requires a software program called an XML Parser or XML**.** Processor. Most XML parsers are available at no charge and for a variety of programming languages (e.g., Java, Perl, C++).

3.4.8 Tutor Marked Assignment:

2. Describe the various types of XML databases
3. Describe the rules for writing XML elements
4. Describe in detail, the purpose of XML

3.4.9 References and Further Reading

Alzarani, H. (2016). Evolution of Object Oriented Database. *Global Journal of Computer Science and Technology: C* .

Ayyasamy, S. B. (May 2017). Performace Evaluation of Native XML database and XML enabled Database. *International Journal of Advanced Research in Computer Science and Software Engineering* .

Balamurugan et al. (2017). Performance Evaluation of Native XML Database and XML Enabled Database. *International Journal of Advanced Research in Computer Science and Software Engg. 7(5),* , 182-191.

Bertino, E. J. (1995). Database security: research and practice. Information systems,.

Gehrke, R. a. (2014). *Database Management Systems, 3ed,* .

George Papamarkos, L. Z. (2010). *XML Database.*

Goebel, V. (2011). *Distributed Database Systems* .

Jorge., D. C. (July 2015). *Basic Principles of Database Security Article* .

Lapis, G. (2005.). *XML and Relational Storage-Are they mutually exclusive* . . IBM Corporation.

Megha, P. T. (2014). An overview of distributed database. *International Journal of Information and Computation Technology.* , 207-214.

Morris, C. C. (2016). *Database System- Implementation and Managemet.*

Morris, C. C. (2017). *DATABASE SYSTEMS.*

N. Derrett, W. K. (1985). Some aspect of oprations in an object oriented database "Database Engineerimg". *IEEE COMPUTER SOCIETY* .

Navathe, R. E. (2010). *FUNDAMENTALS OF Database System: Sixth Edition.* Addison-Wesley.

Özsu, M. T. (June 2014). *Distributed Database Systems.*

T. Atwood. (1985). *"An Object-Oriented DBMS for Design Support Application.*